Technical Specification Document

Birla SCM React Application V2

Version: 0.1.0

Document Date: November 04, 2025

Project Type: Supply Chain Management System

Table of Contents

1. Project Overview
2. Technology Stack
3. Architecture Overview
4. Module Specifications
4.1 Authentication & Authorization Module
4.2 Dashboard Module
4.3 Sales Order Package Module
4.4 Delivery Package Module
4.5 Finance/Invoice Package Module
4.6 Good Receipt Package Module
4.7 Inventory Package Module
4.8 Damage Package Module
4.9 Report Module
4.10 User Administration Module
4.11 SQL Data Maintenance Module
4.12 Dealer Request Module
4.13 Diversion Module
4.14 Profile Module
5. Shared Modules & Services
5.1 Services Module
5.2 Custom Hooks Module
5.3 Redux State Management
5.4 Common Functions
5.5 Shared Components
6. API Endpoints
7. Data Flow
8. Security Features
9. Deployment Configuration

1. Project Overview

Birla SCM React Application V2 is a comprehensive Supply Chain Management (SCM) system built using React and Redux. The application provides end-to-end functionality for managing sales orders, deliveries, inventory, invoicing, and various other supply chain operations specific to the depot and distribution network.

Key Characteristics:
• Enterprise-level React application with modular architecture
• Redux-based state management for complex data flows
• Integration with SAP backend systems via RFC calls
• Supabase integration for additional backend operations
• Role-based access control and user management
• Comprehensive reporting and analytics capabilities
• Mobile-responsive design with Bootstrap
• Export functionality for Excel reports

Target Users:
• Supply Chain Managers
• Depot Administrators
• Sales Order Processors
• Delivery Coordinators
• Finance Personnel
• Inventory Controllers
• System Administrators

2. Technology Stack

Component	Technology	Version
Frontend	React	16.13.1
State Management	Redux with Redux	Thunk
Routing	React Router DOM	5.2.0
UI Framework	React Bootstrap	1.0.1
HTTP Client	Axios	0.21.1
Build Tool	React Scripts	3.4.1
Backend Integration	Supabase	2.44.4

3. Architecture Overview

Application Architecture:

The application follows a standard React-Redux architecture pattern with clear separation of concerns:

1. Presentation Layer (Components)
• React components organized by functional modules

• Lazy loading for performance optimization
• Bootstrap-based responsive UI
• Reusable shared components

2. State Management Layer (Redux)
• Centralized store using Redux
• Reducers for Auth, Loader, and Delivery states
• Redux Thunk for async operations
• Redux Logger for debugging (development)

3. Service Layer
• Axios-based API communication
• Middleware for request/response handling
• Centralized API endpoint management
• Local storage service for session management

4. Routing Layer
• React Router DOM for navigation
• Private routes with authentication guards
• Role-based route access control
• Lazy-loaded route components

5. Backend Integration
• SAP RFC integration for core operations
• Supabase for additional backend services
• RESTful API architecture
• Token-based authentication

Data Flow:
User Interaction → React Component → Redux Action → API Service → Backend (SAP/Supabase) → Redux Reducer → Component Re-render

4. Module Specifications

Authentication & Authorization Module

Description: Handles user authentication, login, and session management
Location: src/components/login

- ***Login Component***

  File: Login.js
  Purpose: User authentication interface
  Features:
  • User credential validation
  • Token-based authentication
  • Session persistence using localStorage
  • Auto-login on token presence
  Dependencies: Redux Auth Actions, API Service
  API Endpoints:
  • /login/login
  • /login/auto_auth

Dashboard Module

Description: Main dashboard interface and navigation system
Location: src/components/dashboard

- ***Wrapper Component***

  File: wrapper.js
  Purpose: Main layout wrapper containing navigation and routing
  Features:
  • Route management for all modules
  • Sidebar navigation integration
  • Navbar integration
  • Protected route handling

- ***Sidebar Component***

  File: SideBar.js
  Purpose: Side navigation menu
  Features:
  • Module-wise navigation
  • Role-based menu visibility
  • Active route highlighting

- ***Navbar Component***

  File: Navbar.js
  Purpose: Top navigation bar
  Features:
  • User profile display
  • Logout functionality
  • Quick actions

Sales Order Package Module

Description: Complete sales order management system
Location: src/components/Sales_Order_Package

- ***Sales Order Form***

  File: SalesOrderForm.js
  Purpose: Create new sales orders
  Features:
  • Multi-step order creation
  • Customer selection (Sold-to and Ship-to party)
  • Material selection with plant availability
  • Shipping type and point configuration
  • Sales area selection
  • Incoterm and transport zone management
  • Order validation and submission
  API Endpoints:
  • /rfc/get_orders_type_dropdown
  • /rfc/get_zfm_plants
  • /rfc/get_shipping_type
  • /rfc/get_sold_to_party
  • /rfc/get_ship_to_party
  • /rfc/get_materials_of_plants
  • /rfc/create_sales_order

- ***Sales Order List***

  File: SalesOrderList.js
  Purpose: View and manage existing sales orders
  Features:
  • Order listing with filters
  • Search functionality
  • Pagination support
  • Order details view
  • Edit capability
  • Export to Excel

- ***Sales Order Edit***

  File: SalesOrderEdit.js
  Purpose: Modify existing sales orders
  Features:
  • Fetch order details
  • Modify order items
  • Update quantities and materials
  • Rejection reason handling
  • Order status update
  API Endpoints:
  • /fetch-so-details
  • /update-so-details

- ***Search Dialogue***

  File: SearchDialogue.js
  Purpose: Advanced search interface for orders
  Features:
  • Multi-criteria search
  • Filter by date range
  • Filter by customer
  • Filter by material

Delivery Package Module

Description: Delivery order creation and management
Location: src/components/Delivery_Package

• *Delivery Create*

File: DeliveryCreate.js
Purpose: Create delivery orders from sales orders
Features:
• Link to sales orders
• Storage location selection
• Loading point configuration
• Available stock verification
• Transporter assignment
• Vehicle number entry
• Delivery quantity management
API Endpoints:
• /get-order-details
• /get-storage-location
• /get-loading-points
• /get-available-stock
• /get-transporter
• /create-delivery

• *Delivery List*

File: DeliveryList.js
Purpose: View all delivery orders
Features:
• Delivery listing
• Status tracking
• Filter and search
• Export functionality
API Endpoints:
• /delivery-list

• *Delivery Edit*

File: DeliveryEdit.js
Purpose: Modify delivery orders
Features:
• Edit delivery details
• Update quantities
• Change transporter
• Modify vehicle details
API Endpoints:
• /get-delivery-details-for-edit
• /update-delivery

- ***Post Goods Issue***

  File: DORFC.js
  Purpose: Create Post Goods Issue (PGI)
  Features:
  • PGI creation from delivery
  • SAP integration
  • Stock movement posting
  API Endpoints:
  • /create-pgi

Finance/Invoice Package Module

Description: Financial operations and invoice management
Location: src/components/Finance_Package

- ***Invoice Create***

  File: FinanceInvoiceCreate.js
  Purpose: Generate invoices from deliveries
  Features:
  • Link to delivery orders
  • Invoice generation
  • Tax calculations
  • Customer billing details
  • Payment terms
  API Endpoints:
  • /create-invoice
  • /get-delivery-details

- ***Invoice List***

  File: FinanceInvoiceList.js
  Purpose: Manage all invoices
  Features:
  • Invoice listing
  • Invoice search
  • Print functionality
  • PDF generation
  • Invoice status tracking
  API Endpoints:
  • /invoice-list
  • /print-invoice

- ***Customer Search***

  File: SearchCustomersFrom.js, SearchCustomersTo.js
  Purpose: Search and select customers
  Features:
  • Customer lookup
  • Region-wise filtering
  • Customer details display
  API Endpoints:
  • /search-customer

Good Receipt Package Module

Description: Goods receipt and inventory receiving management
Location: src/components/GoodReceipt_Package

- ***Good Receipt Create***

  File: GoodReceiptCreate.js
  Purpose: Record goods receipt
  Features:
  • Receipt documentation
  • Condition type selection
  • Storage location assignment
  • Receiving plant configuration
  • Quality inspection
  • Batch management
  API Endpoints:
  • /initial-good-receipt-list
  • /fetch-condition-type
  • /fetch-condition-based-storage-location
  • /good-receipt-create

- ***Good Receipt Listing***

  File: GoodReceiptListing.js
  Purpose: View all goods receipts
  Features:
  • Receipt listing
  • Filter by plant and date
  • Receipt details view
  • Export to Excel
  API Endpoints:
  • /fetch-receiving-plant-gr-listing
  • /fetch-ship-type-gr-listing
  • /fetch-good-receipt-list

Inventory Package Module

Description: Inventory management and stock operations
Location: src/components/Inventory_Package

- ***Stock Create***

  File: StockCreate.jsx
  Purpose: Manual stock entry and adjustments
  Features:
  • Stock creation
  • Material selection
  • Quantity entry
  • Batch assignment

- ***Display Inventory***

  File: DisplayInventory.jsx
  Purpose: View current inventory levels
  Features:
  • Real-time stock display
  • Material-wise breakdown
  • Plant-wise inventory
  • Storage location details

- ***Approve Inventory***

  File: ApproveInventory.jsx
  Purpose: Inventory approval workflow
  Features:
  • Pending inventory review
  • Approval/rejection
  • Remarks addition

- ***Inventory Reports***

  File: InventoryReportPackage.jsx
  Purpose: Generate inventory reports
  Features:
  • Stock overview reports
  • Movement reports
  • Compliance reports
  • RR Summary reports

- ***Compliance Report***

  File: ComplianceReport.jsx
  Purpose: Regulatory compliance tracking

Features:
• Compliance status
• Audit trail
• Regulatory reports

Damage Package Module

Description: Damage claim and exception management
Location: src/components/Damage_Package

- ***Claim Intimation***

  File: ClaimIntimation.jsx
  Purpose: Report damage claims
  Features:
  • Damage documentation
  • Photo upload
  • Claim amount calculation
  • Insurance linkage

- ***Rake Damage Data***

  File: RakeDamageData.jsx
  Purpose: Track rake-specific damages
  Features:
  • Rake identification
  • Damage details
  • Responsibility assignment

- ***Approve Damage Data***

  File: ApproveDamageData.js
  Purpose: Approve/reject damage claims
  Features:
  • Claim review
  • Approval workflow
  • Settlement processing

- ***Demurrage/Wharfage Data***

  File: DemmurageWharfageData.jsx
  Purpose: Track demurrage and wharfage charges
  Features:
  • Delay tracking
  • Charge calculation
  • Payment processing

Report Module

Description: Comprehensive reporting system
Location: src/components/Report

- ***Sales Register***

  File: SalesRegister.js
  Purpose: Sales transaction reports
  Features:
  • Sales summary
  • Date-wise analysis
  • Customer-wise breakdown
  • Material-wise reports

- ***LE Register***

  File: LeRegister.js
  Purpose: Legal entity register
  Features:
  • Company code filtering
  • Region-wise reports
  • Distribution channel analysis
  API Endpoints:
  • /leregister-fetch-company-code
  • /leregister-fetch-region
  • /leregister-fetch-list

- ***FI Daywise Report***

  File: FIDaywiseReport.js
  Purpose: Financial day-wise reporting
  Features:
  • Daily financial summary
  • Revenue tracking
  • Outstanding reports
  API Endpoints:
  • /fi-daywise-fetch-list

- ***Stock Overview Report***

  File: StockOverViewReport.js
  Purpose: Inventory overview reporting
  API Endpoints:
  • /stock-overview-report

- ***Debit Credit Report***

File: DebitAndCredit.js
Purpose: Debit/credit note reporting
API Endpoints:
• /debit-credit-report-report

- ***Collection Report***

File: CollectionReport.jsx
Purpose: Payment collection tracking

User Administration Module

Description: User and access management
Location: src/components/User_Admin_Package

- ***User Create Form***

  File: UserCreateForm.js
  Purpose: Create new system users
  Features:
  • User details entry
  • Role assignment
  • Permission configuration
  • Plant/depot mapping
  API Endpoints:
  • /login/create

- ***User Edit List***

  File: UserEditList.js
  Purpose: Manage existing users
  Features:
  • User listing
  • Edit user details
  • Deactivate users
  • Password reset
  API Endpoints:
  • /login/allUser
  • /login/update

- ***Give User Permissions***

  File: GiveUserPermissions.js
  Purpose: Manage user permissions
  Features:
  • Module-wise access control
  • CRUD permissions
  • Data visibility controls

- ***CFA Depot Mapping***

  File: CFADepotMap.js, CFADepotUnMap.js
  Purpose: Map users to CFA and depots
  Features:
  • CFA assignment
  • Depot assignment
  • Territory mapping

- ***BHCS Depot Mapping***

  File: BHCSDepotMapUnMap.js
  Purpose: BHCS depot user mapping

SQL Data Maintenance Module

Description: Master data management
Location: src/components/SQL_Data_Maintain

- ***Material Management***

  File: MaterialMangement.js
  Purpose: Manage material master data
  Features:
  • Add new materials
  • Update material details
  • Material category management

- ***Plant Management***

  File: PlantManagement.js
  Purpose: Manage plant master data
  Features:
  • Plant creation
  • Plant details update
  • Plant activation/deactivation

- ***CFA Management***

  File: CFAManagement.js
  Purpose: Manage CFA master data

- ***Incoterm Management***

  File: IncotermManagement.js
  Purpose: Manage shipping terms

- ***GR Material Management***

  File: GRMaterialMangement.js
  Purpose: Goods receipt material configuration

Dealer Request Module

Description: Dealer sales order request management
Location: src/components/Dealer_Request

- ***Dealer Request List***

  File: DealerRequestList.jsx
  Purpose: View dealer SO requests
  Features:
  • Request listing
  • Status tracking
  • Approval/rejection
  API Endpoints:
  • /login/openapi/get_requests

- ***Dealer SO Create***

  File: DealerSOCreate.jsx
  Purpose: Create SO from dealer request
  Features:
  • Request to SO conversion
  • Auto-populate from request
  • SO creation
  API Endpoints:
  • /login/openapi/update_so_request

Diversion Module

Description: Handle shipment diversions
Location: src/components/Diversion

- ***Diversion Management***

  File: Diversion.js
  Purpose: Manage shipment route changes
  Features:
  • Change delivery destination
  • Update shipping details
  • Approval workflow

Profile Module

Description: User profile management
Location: src/components/Profile_Package

- ***User Profile***

  File: UserProfile.js
  Purpose: View and edit user profile
  Features:
  • Profile display
  • Update personal details
  • Change password
  • View assigned depots/plants
  API Endpoints:
  • /login/user_data
  • /login/update_pwd

5. Shared Modules & Services

Services Module

Location: src/services

- **API Service** (apicall.js)
  Centralized API communication
  Features: Axios wrapper, Request interceptors, Response handling, Error handling, Token management
- **API Endpoints** (apis.js)
  API endpoint definitions
  Features: Centralized endpoint management, Environment-based URLs, API versioning
- **Local Storage Service** (localStorage.js)
  Browser storage management
  Features: Token storage, User data caching, Session management
- **Route Decision Service** (decideRoute.js)
  Route authorization logic
  Features: Role-based routing, Permission checking, Route redirection
- **Utility Service** (utils.js)
  Common utility functions
  Features: Data formatting, Validation helpers, Common calculations

Custom Hooks Module

Location: src/hook

- **useRegion** (useRegion.js)
  Region data hook
- **usePlant** (usePlant.js)
  Plant data hook
- **useDepot** (useDepot.js)
  Depot data hook
- **useCFA** (useCFA.js)
  CFA data hook
- **useCustomerRegionWise** (useCustomerRegionWise.js)
  Customer filtering by region

Redux State Management

Location: src/reducers, src/actions

- **Auth Reducer** (reducers/authreducer.js)

Authentication state management
State: isAuthenticated, user, token

- **Loader Reducer** (reducers/loaderreducer.js)
  Global loading state
  State: loading
- **Delivery Reducer** (reducers/deliveryreducer.js)
  Delivery data state management

Common Functions

Location: src/Functions

- **Excel Export** (XLSXExport.jsx, ExcelReport.jsx)
  Export data to Excel
- **File Upload** (FileUpload.jsx)
  Handle file uploads
  APIs: /upload, /image
- **Filter Functions** (filterData.js, filterDataReport.js)
  Data filtering utilities
- **Fetch Functions** (fetchCustomer.js, fetchPlant.js, fetchTransporter.js)
  Data fetching utilities
- **Supabase Integration** (SupaBase.js)
  Supabase backend operations

Shared Components

Location: src/components/shared

- **React Table** (ReactTable.jsx)
  Reusable data table component

Utilities

Location: src/components/utils

- **Private Route** (PrivateRoute.js)
  Protected route wrapper
- **Login Wrapper** (LoginWrapper.js)
  Login route wrapper
- **Loader** (loader.js)
  Loading indicator component

8. Security Features

Authentication & Authorization:
• Token-based authentication using JWT
• Session management via localStorage
• Auto-logout on token expiration
• Password encryption and secure storage
• Role-based access control (RBAC)

Route Protection:
• Private route wrapper for authenticated pages
• Login wrapper for public routes
• Automatic redirect to login on unauthorized access
• Permission-based route visibility

API Security:
• Request/response interceptors
• Token injection in API headers
• Error handling for unauthorized requests
• HTTPS communication
• App ID and App Secret for API authentication

Data Security:
• Input validation on all forms
• XSS protection
• CSRF token handling
• Secure data transmission

User Management:
• Password complexity requirements
• User activity logging
• Role and permission management
• Plant/depot-level data access control

9. Deployment Configuration

Build Configuration:
• Production build: yarn build
• Development server: yarn start
• Testing: yarn test

Environment Variables:
• NODE_ENV: development/production
• BASE_URL: https://bcldvportal.birlacorp.com/depot/
• API endpoints configured in src/services/apis.js

Production Deployment:
• Build artifacts in 'build' folder
• Static file serving
• Minified and optimized bundles
• Hash-based cache busting

Browser Support:
Production:
• >0.2% market share
• Not dead browsers
• Not Opera Mini

Development:
• Latest Chrome version
• Latest Firefox version
• Latest Safari version

Performance Optimizations:
• Lazy loading of route components
• Code splitting
• React.Suspense for async components
• Global loader for better UX
• Memoization for expensive computations